



第六章 模块与包

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院



目录

1. 模块
2. 包
3. 应用案例
4. 本章习题



模块与包

- 随着软件任务变得日益复杂，编写的程序代码量也急剧增加，因此**有效地管理这些代码变得尤为重要**
- 在许多情况下，代码的开发是一个团队协作的过程，这就要求必须考虑**团队成员之间的协同性**
- 借鉴软件工程中的“**分而治之**”原则，一个大的软件项目可以被划分成多个小的单元，这些单元即为**模块**，而相关联且功能紧密的多个模块根据它们的物理或逻辑关系被组织在一起，形成了**包**
- 这样的代码组织方式不仅有利于**功能划分**，也促进了**代码复用**和**便于维护**



1. 模块

- **函数与模块**

- **函数的重用**：函数提供精细的代码重用机制，其方法是：通过导入函数定义。
- **模块的重用**：模块包含多个函数或变量，整体导入，提高效率，简化重用（如绘图函数集）。
- **示例**：一次性导入绘图模块，使用所有相关功能。

- **模块的价值**

- **代码组织**
 - **模块作为功能容器**：封装函数、类和变量
 - **划分**：逻辑上独立的部分，专注特定任务
- **可维护性与模块化**
 - **提升代码可读性和重用性**
 - **减少名称冲突，简化团队协作**

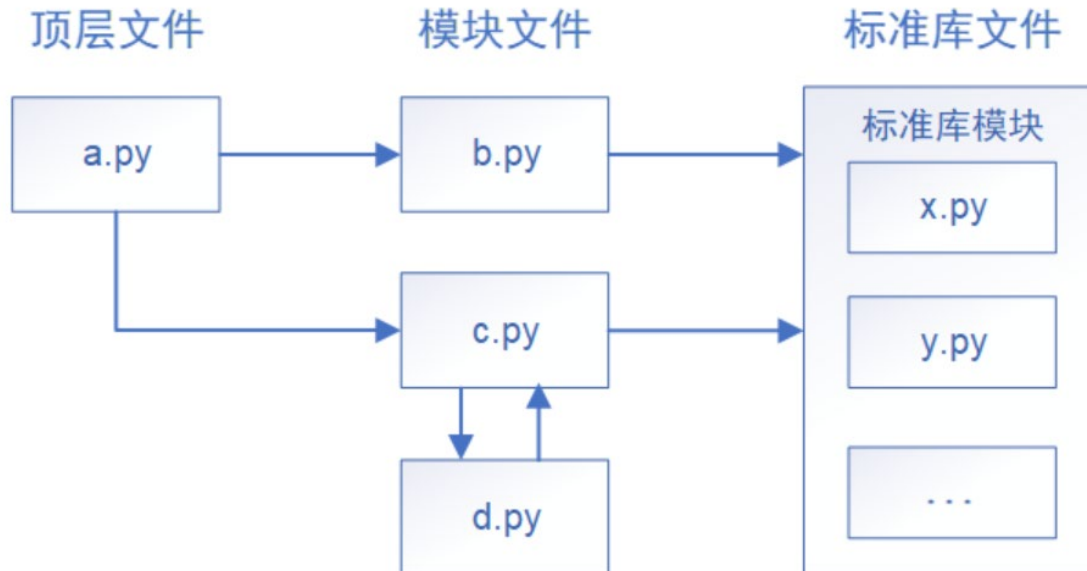


Python模块

- 在Python中，一个模块实际上就是一个拥有 `.py` 后缀的文件。这意味着每个Python文件本身都可以被视为一个模块，可以被其他Python代码导入和使用
- 假设有一个名为`math_operations.py`的模块，它定义了一些基本的数学运算：
 - `# math_operations.py`
 - `def add(x, y):`
 - `return x + y`
 - `def subtract(x, y):`
 - `return x - y`
 - `def multiply(x, y):`
 - `return x * y`
 - `def divide(x, y):`
 - `return x / y`
- 在另一个文件中，可以导入这个模块，并使用其中定义的函数：
 - `# main.py`
 - `import math_operations`
 - `sum_result = math_operations.add(10, 5)`
 - `product_result = math_operations.multiply(10, 5)`



- 程序结构通常由多个互相协作的文件构成
 - 包括一个主要的入口文件以及多个模块文件
 - 主入口文件负责定义整个程序的主要流程，它通过调用其他模块文件中的函数来执行各种特定的任务
 - 这些模块文件又可能相互调用，形成一个功能丰富的网络
 - 模块文件还可以利用Python的内置标准库来扩展其功能





模块的导入与使用

- **Python有两种主要的模块导入方法：**导入整个模块和从模块中导入特定对象
- **1. 导入整个模块：**使用**import关键字**导入一个完整的模块，使得当前文件可访问模块中定义的所有函数和类
 - **语法结构**
 - `import 模块名 [as 别名]` #导入模块并起一个别名
 - `模块名.对象名` # 访问该模块中的对象
 - `别名.对象名` # 作用同上一句
 - **举例：**
 - `import math`
 - `print(math.sqrt(16))` # 输出: 4.0
 - `import numpy as np` #导入numpy模块，并定义别名np
 - `np.array([1, 2, 3])` #调用np中的array对象



• 2. 从模块中导入特定对象

- 如果只需要使用模块中的特定函数或类，可以选择只导入需要的部分。这样做有助于节省内存，提高程序效率
- **语法结构：**
 - `from 模块名 import 对象名 [as 别名]`
- **举例：**
 - `from math import sqrt`
 - `print(sqrt(16))` # 导入sqrt对象后直接使用，不用加限定的模块名. 输出: 4.0



模块循环引用问题

- 在Python中，**两个模块相互引用**是可能的，但这种情况**需要小心处理**，否则容易引起循环引用问题，特别是在模块级别的代码执行时
 - 循环引用主要发生在两个或更多的模块互相导入对方，这可能导致导入失败或者运行时错误，因为Python解释器在处理导入语句时可能遇到尚未完全初始化的模块
- **如何解决或避免循环引用**
 - **1. 重新设计代码结构：**
 - 通常，循环引用表明可能需要重新考虑模块的设计。尽量避免模块间的直接依赖，或者将相互依赖的部分提取到一个单独的模块中
 - **2. 使用局部导入：**
 - 将导入语句放在函数内部或者需要它的地方，而不是文件的顶部。这样，只有在实际需要模块功能的时候才进行导入，可以避免初始化阶段的循环依赖问题
 - **3. 延迟导入：**
 - 对于类定义中可能出现的循环引用，可以在**类的方法内部进行导入**，从而延迟导入的时机



示例

- 假设有两个模块，moduleA 和 moduleB，并且它们需要相互引用

```
# moduleA.py
from moduleB import B
class A:
    def foo(self):
        print("A foo")
        b = B()
        b.bar()
```



```
# moduleA.py
class A:
    def foo(self):
        from moduleB import B
        print("A foo")
        b = B()
        b.bar()
```

```
# moduleB.py
from moduleA import A
class B:
    def bar(self):
        print("B bar")
        a = A()
        a.foo()
```



```
# moduleB.py
class B:
    def bar(self):
        from moduleA import A
        print("B bar")
        a = A()
        a.foo()
```



模块的 `__name__`

- 在Python中，每个模块都包含一个内置的属性 `__name__`
 - 若模块是被直接运行，`__name__`的值被设为 `__main__`
 - 若它是被导入到另一个模块中，`__name__`则会被设为该模块的文件名（不包括.py扩展名）。
 - 该属性的特性可用来控制模块的行为，依据模块是作为独立程序运行还是被导入使用。
 - `if __name__ == "__main__":`
 - `# 该代码块仅在模块直接运行时执行`
 - `main_function()`
- `__name__`属性适合于在模块中进行单元测试或者将其作为独立脚本执行
 - `def add(a, b):`
 - `return a + b`
 - `if __name__ == "__main__": # 这些测试代码只在模块被直接运行时执行`
 - `result = add(2, 3)`
 - `if result == 5:`
 - `print("测试通过")`
 - `else:`
 - `print("测试未通过")`



- **模块通常由三部分组成：**

- **函数和类定义**：这些是模块的主要组成部分，用于定义可重用的函数和类
- **顶层代码**：这是模块的主体部分，可以包含函数调用、类实例化等操作
- **可执行语句**：位于模块底部，通常用于测试。这部分代码应当放在 `if __name__ == "__main__":` 块中，以确保只有在模块被直接运行时才执行

- **模块的命名和组织**

- **命名**：模块名应该简洁且描述性强。通常使用小写字母，多个单词间可用下划线分隔
- **组织**：将相关的函数和类组织到同一个模块中。例如，所有与数学相关的函数可以放在一个名为 `math_utils.py` 的模块中



- 编写一个简单的模块

- `# greetings.py`
- `# 函数和类定义`
- `def say_hello(name):`
- `return f"Hello {name}!"`
- `def say_goodbye(name):`
- `return f"Goodbye {name}! "`
- `# 顶层代码`
- `print("Greetings module is loaded.")`
- `# 可执行语句`
- `if __name__ == "__main__":`
- `# 测试模块功能`
- `print(say_hello("Alice"))`
- `print(say_goodbye("Bob"))`



6.2 包

- 多个变量和运算构成了函数，多个函数和全局变量构成了模块，那么多个模块构成了包 (package)
- 在Python中，一个包可看作包含多个模块的目录，它通过文件夹和文件的形式组织模块，并允许创建模块的层次结构
- Python中引入包的概念是符合模块化的设计思想，将多个相关的模块放在同一个包中，便于管理和查询，可大幅简化文件搜索路径设置，也使得程序更易于理解



创建包

- 从包的基本结构来讲，包实际上是一个文件目录，用来包含多个相关的模块文件
 - 每个包目录下必须有一个名为 `__init__.py` 的Python文件，用来标识该目录为Python的一个包，即使该文件是一个没有任何内容的空文件
 - 例如，在一个网上商城的应用中，可能会创建一个名为 `ecommerce` 的包，它包含了各种管理在线交易的模块
 - `ecommerce/`
 - `├── __init__.py`
 - `├── shopping_cart.py`
 - `├── payments.py`
 - `├── products.py`
 - `└── customers.py`



使用包

- 当需要在程序中使用包中的模块时，可使用import导入语句导入某一个包中的模块
- **语法结构**
 - `from 包名 import 模块名`
 - 例如，要导入上面例子中的 shopping_cart 模块：
 - `from ecommerce import shopping_cart`
 - 或者，若只需要使用这个模块中的特定函数或类：
 - `from ecommerce.shopping_cart import add_to_cart`



绝对导入

- 根据 PEP 8 规范，在 Python 中导入模块时，**推荐使用绝对导入而不是相对导入**
- **绝对导入：**绝对导入使用完整的路径从项目的根目录开始引用模块。这种方法明确了模块在项目中的位置，使得代码更容易理解，尤其是在大型项目中
 - my_project/
 - |— main.py
 - |— mypackage/
 - | |— __init__.py
 - | |— submodule1.py
 - | |— submodule2.py
 - 若想在 main.py 中导入 mypackage 下的 submodule1：
 - # main.py
 - import mypackage.submodule1
- **相对导入：**使用当前模块的位置作为参考点来导入其他模块。使用点 (.) 符号表示当前目录
 - # submodule2.py
 - from . import submodule1 # 从当前包中导入名为 submodule1 的模块



3. 应用案例：经典的包分层结构

- 在Python项目开发中，一个经典且广泛使用的包结构通常包括若干核心组件，如模块、子包、测试代码、安装脚本、文档等

```
• my_project/
• |
• ├── my_project/      # 主包目录
• |   ├── __init__.py  # 初始化脚本，定义包的导入行为
• |   ├── module1.py   # 主包的模块
• |   ├── module2.py   # 另一个主包的模块
• |   ├── subpackage1/  # 子包
• |   |   ├── __init__.py # 子包的初始化脚本
• |   |   ├── submodule1.py # 子包内的模块
• |   |   └── submodule2.py # 另一个子包内的模块
• |   └── subpackage2/
• |       ├── __init__.py
• |       ├── submodule1.py
• |       └── submodule2.py
• |
• ├── tests/           # 测试目录
• |   ├── __init__.py
• |   ├── test_module1.py # 测试模块1
• |   ├── test_module2.py # 测试模块2
• |   ├── test_submodule1.py # 测试子模块1
• |   └── test_submodule2.py # 测试子模块2
• |
• ├── docs/            # 文档目录
• |   ├── conf.py       # Sphinx 文档配置文件
• |   ├── index.rst     # 主文档文件
• |   └── ...           # 其他文档文件
• |
• ├── setup.py          # 安装脚本
• ├── README.md         # 项目README文件
• └── LICENSE            # 许可证文件
```

- my_project: 这是顶层包目录，所有的源代码都在这个目录下。
- __init__.py: 这个文件使Python将包含它的目录视为一个包。它可以为空或者定义包的初始化代码。
- module1.py, module2.py: 主包中的模块文件。
- subpackage1/, subpackage2/: 包内的子包，每个子包可以包含自己的模块和子包。
- tests/: 包含所有单元测试。测试通常以`test\_`开头命名。
- docs/: 存放项目文档的目录，常使用Sphinx生成文档。
- setup.py: 安装脚本，定义了包的依赖关系和元数据，用于发布到PyPI等。
- README.md: 项目的简介文件，通常包含项目说明、安装和使用指南。
- LICENSE: 项目的许可证文件，说明项目的使用和分发条件。



案例：机器学习包的分层结构

- **机器学习**是一个应用数学和统计学方法，使计算机系统利用数据进行学习和模式识别的研究领域。它通过建立算法模型，基于提供的数据进行预测或决策
 - **分类**：这类算法将数据分入预先定义的标签或类别，通常用于预测属于哪个类别的任务
 - **聚类**：这类算法尝试将数据集中的样本分组，使得组内的样本比其他组的样本更相似，常用于探索性数据分析
 - **回归**：回归算法预测连续值的输出，如房价或销售量等



- **机器学习包的示意结构：**

- machinelearning/
 - |
 - |— __init__.py
 - |— classifier/
 - | |— __init__.py
 - | |— knn.py # K最近邻算法
 - | |— decision_tree.py # 决策树算法
 - | |— svm.py # 支持向量机算法
 - | |— random_forest.py # 随机森林算法
 - |— clustering/
 - | |— __init__.py
 - | |— k_means.py # K均值聚类算法
 - | |— dbscan.py # DBSCAN聚类算法
 - |— regression/
 - | |— __init__.py
 - | |— linear_regression.py # 线性回归算法
 - | |— ridge_regression.py # 岭回归算法
- **导入和使用包中的模块**
- `from machinelearning.classifier import knn`



本章习题

• 一、理论知识

1. 解释Python中模块和包的区别。
2. 描述如何在Python中创建一个模块。
3. 列出在Python程序中导入模块的两种主要方式，并说明它们的区别。
4. 说明__name__属性在Python模块中的作用。
5. 讨论绝对导入和相对导入在Python中的区别及应用场景。
6. Python标准库提供了哪些常用模块？举例说明。
7. 解释什么是包的层次结构，以及为什么它对大型项目很重要。

• 二、编程实践

1. 给定一个模块 math_operations.py，该模块包含多个数学运算函数。编写代码导入此模块，并调用名为 multiply 的函数。
2. 创建一个包含主函数和其他几个函数的Python文件。使用 if __name__ == "__main__": 条件，确保主函数只在该文件被直接运行时执行。



The End !